

Are Edges Topology or Supervision: Edge-aware Message Passing for Graph based Recommendation

Jiarui Jin^{1,*}, Yangkun Wang^{2,*}, Weinan Zhang¹, Quan Gan³, Xiang Song³, Yong Yu¹, Zheng Zhang³, David Wipf³

¹Shanghai Jiao Tong University, ²University of California San Diego, ³Amazon
{jinjiarui97,wnzhang,yyu}@sjtu.edu.cn,yaw048@ucsd.edu,{quagan,xiangsx,zhaz,daviwipf}@amazon.com

ABSTRACT

Many recent investigations that leverage (heterogeneous) graphs in recommender systems to characterize diverse entities and their complex relations, formulate the recommendation as the edge prediction (a.k.a., link prediction) task. With the prevalence of graph neural networks (GNNs), many existing graph based recommendation methods are following the paradigm of message passing neural networks (MPNNs) originally proposed for the node classification task. However, these works lack elaborated designs regarding the distinctions between the node classification and edge prediction tasks: (i) the edges only constitute *topology* in the node classification task but can be used as both *topology and supervisions* (i.e., labels) in the edge prediction task; (ii) the node prediction is made over each *individual node*, while the edge prediction is determined by each *pair of nodes*. To this end, we propose a novel (heterogeneous) graph based recommendation paradigm named Edge-aware Message Passing based REcommendation (EMPIRE), where we first introduce an *edge splitter* to specify the use of training edges where each edge is solely used as either topology or supervision (named as topology edge or supervision edge). Based on the splitted edges, we develop a novel *edge-aware message passing* mechanism that generates the messages to the source nodes from their neighbor nodes (through topology edges) being aware of the target nodes (through supervision edges). Based on the above message generation, we further design an extension to allow our model to focus on the differences between node pairs connected by the supervision edges and node pairs unconnected. To effectively sample these unconnected node pairs, we use the common-neighbor heuristics to define “hard” negative node pairs and present corresponding sampling algorithms. Experimental results verify that our method can be seamlessly incorporated into various MPNN architectures and consistently improve their performance on multiple homogeneous and heterogeneous graph based recommendation benchmarks.

KEYWORDS

Edge-aware Message Passing, Graph Neural Network, Link Prediction

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Pre-print, Under review., 2023

© 2023 Association for Computing Machinery.
ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00
<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

1 INTRODUCTION

Graphs especially heterogeneous graphs (a.k.a., heterogeneous information networks) [22, 23] as a common structure for complex relational data, have been widely used to organize the recommendation data. Due to the remarkable success of graph convolution networks (GCNs) [17], many advanced graph neural networks (GNNs) (e.g., graph attention networks (GAT) [27], GraphSAGE [8]) designs have been proposed regarding the node classification task. However, (heterogeneous) graph based recommendation methods formulate the recommendation task as the edge prediction (a.k.a., link prediction) task, whose goal is to predict the edge existence of each user-item pair. Many prevailing solutions* (e.g., HAN [31] and HetGNN [39]) are directly extended from existing GNN frameworks especially message-passing neural networks (MPNNs) [7] through plugging the representation vectors of each pair of user and item nodes into a pre-defined scoring function.

We argue that as MPNNs are originally designed for the node classification task instead of the edge prediction task, this paradigm not paying enough attention to the differences between these tasks would lead to limited performance.

- *Given a graph, in the node classification task, the supervisions (i.e., labels) are on the nodes and the edges are only used to construct the topology (i.e., graph structure); while in the edge prediction task, the edges are required to be used as both the topology and the supervisions.* As a result, the topology in the node classification task remains the same during the training and test phases. In contrast, in the edge prediction task, to avoid the label leakage issue, test edges are removed before training, resulting in different topologies during the training and test phases. Since the performance of MPNNs largely depends on the topology [25], existing approaches are likely to *over-fit the training graph* and would cause *low generalizability to the test graph*.
- *The classification of each node is solely determined by the property of the node; while the prediction over each node pair depends on the properties of two nodes.* We note that MPNNs aggregate the neighborhood of each node, and directly applying them to the edge prediction would make the message passing to the source nodes *invariant to the target nodes*. However, one of the main challenges in personalized recommendations is to distinguish and exploit informative neighbor nodes. Taking Figure 1 as an

*Work done during internship at Amazon Web Services Shanghai AI Lab.

*We notice that there is another line of edge prediction frameworks (e.g., SEAL [40]) that incorporate MPNNs by formulating the edge prediction as the subgraph classification problem. However, generally defining subgraph patterns [37] and efficiently sampling subgraphs [6, 16] is non-trivial, especially in the context of heterogeneous graph data, which causes limited use for this series of work. Since our paper mainly focuses on (heterogeneous) graph based recommendations, we consider this family of works out of the scope of this paper.

example, when predicting the edge existence between User A and Item A, in most cases, we are seeking the supporting evidence for the edge existence, as most graphs are sparse. In this regard, the message from Type A to User A would be more informative than one from Type B; because (i) Type A connects User A and Item A and thus is supportive to the existence, (ii) Type B is a neighbor node of User B that might lead to unconnected node pair $\langle \text{User B}, \text{Item A} \rangle$ and thus is not likely to be supportive.

In this paper, we propose a novel recommendation paradigm based on (heterogeneous) graphs called Edge-aware Message Passing REcommendation (EMPIRE), which elaborates on the use of each training edge and tailors the message passing mechanism to make it a good fit for the edge prediction on (heterogeneous) graphs.

To achieve this, we first elaborate on the use of training edges by splitting edges into two subsets, named topology and supervision edges. As illustrated in Figure 1, for each round, we use the topology edges (e.g., $\langle \text{User A}, \text{Type A} \rangle$) to construct the training graph, and randomly sample unconnected node pairs (e.g., $\langle \text{User B}, \text{Item A} \rangle$) as the negative instances together with the supervision edges (e.g., $\langle \text{User A}, \text{Item A} \rangle$) as the positive instances to constitute the *supervision set* which is used for training. As a result, each training edge is used as either the topology or the supervision. Also, our *edge splitter operating on the training edges* could align the training and test procedures: using a partly-observed graph to predict the edge existence of given node pairs; and thus can improve the generalizability of graph based recommendation model.

On the basis of the splitted edges, in order to make the messages to the source nodes (through topology edges) being aware of the target nodes (through supervision edges), we propose a new *edge-aware message passing mechanism* to use the target nodes to attend the messages to the source nodes from their neighborhoods (e.g., to high-weight the message from Type A to User A when predicting the node pair $\langle \text{User A}, \text{Item A} \rangle$ in Figure 1). We further notice that after each split, there are three types of node pairs: ones connected by the topology edges, ones in the supervision set, and unconnected ones. To distinguish the positive node pairs from the negative ones, MPNNs are required to pay special attentions to the differences between their neighborhoods. For example, when considering node pairs $\langle \text{User A}, \text{Item A} \rangle$ and $\langle \text{User B}, \text{Item A} \rangle$, we notice that the messages from Type B should be light-weighted, as Type B appears in the neighborhoods of both User A and User B and thus is not likely to be informative for distinguishing.

In addition, we notice that the negative instances play an important role in the final edge prediction. Inspired by [38] which reveals that “hard” negative samples would improve the effectiveness and stability of the information retrieval models, we define the unconnected node pairs satisfying the common-neighbor heuristics as “hard” negative samples in the edge prediction task. The underlying reason is that the non-parametric common neighbor method [1] is a strong baseline regarding the edge prediction task, and thus it would be hard to distinguish unconnected node pairs with the common neighbors and connected node pairs. However, as discussed in [25], finding all these node pairs is much time-consuming. To address this issue, we propose a new negative node-pair sampling trick, which allows us to efficiently sample or retrieve these node pairs as the negative samples.

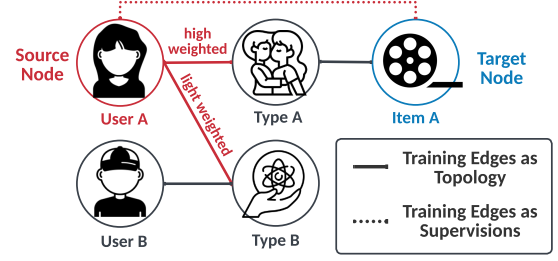


Figure 1: An illustrated example of EMPIRE, where given the training graph, when predicting node-pair $\langle \text{User A}, \text{Item A} \rangle$, (i) the use of each training edge should be specified: as supervisions (shown as dashed lines) or topology (shown as solid lines); (ii) messages to source node User A from its neighbor node Type A should be high-weighted as it connects User A and Item A (as supportive evidence); (iii) messages to source node User A from its neighbor node Type B should be light-weighted as it appears in unconnected node pair $\langle \text{User B}, \text{Item A} \rangle$ (as unsupportive evidence).

In summary, the contributions of this paper are three-fold:

- We reveal that edges simultaneously used as topology and supervisions would encourage MPNN models to over-fit the inputted training graph, and thus hold low generalizability to test graph.
- We develop an edge splitter and design a novel edge-aware message passing mechanism based on the splitted edges to make messages to the source nodes conditioned on the target nodes.
- We present a new negative node-pair sampling trick that efficiently samples “hard” node pairs satisfying the common-neighbor heuristics to construct the negative instances.

Our empirical evaluation demonstrates that EMPIRE can be seamlessly adapted to multiple MPNN architectures-based recommendation frameworks and consistently lead to significant improvements.

2 PRELIMINARIES

2.1 Problem Definition and Related Work

We begin by describing the formation of (heterogeneous) graph-based recommendations and introducing the associated notations.

DEFINITION 1. (HETEROGENEOUS GRAPH-BASED RECOMMENDATIONS) Given a tuple $(\mathcal{U}, \mathcal{I}, \mathcal{A}, \mathcal{R})$ where $\mathcal{U} = \{u_1, \dots, u_p\}$ denotes a set of p users, $\mathcal{I} = \{i_1, \dots, i_q\}$ denotes a set of q items, $a \in \mathcal{A}$ denotes an attribute associated with objects (e.g., Type in Figure 1), and $r \in \mathcal{R}$ represents the relation (e.g., click) between different types of objects. We can construct a heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ where $\mathcal{V} = \mathcal{U} \cup \mathcal{I} \cup \mathcal{A}$ is a set of nodes with multiple types and $\mathcal{E}$ is a set of edges with multiple relations $\mathcal{R}$. Then, the goal is to predict the edge existence between each user-item node pair $\langle u_s, i_t \rangle$ where u_s is the source (user) node and i_t is the target (item) node.

To accomplish this task, the key technique is to learn the representative vector for each object, which has been widely investigated in previous graph representation learning work. Also, another surge of investigations is working on how to leverage the representation learning methods into the recommendation task.

MPNN-based recommendation investigates how to leverage graph structure (especially heterogeneous graphs) to improve the recommendation performance through GNNs, especially MPNNs. One line of research [2, 4, 13, 14, 21, 24, 30, 35, 36] focuses on defining the neighborhoods and integrates the structural information to enhance the representations of user and item from their neighborhood. For example, Wang et al. [31] (HAN) used a hierarchical attention network to learn the importance of metapath based neighbors and different metapaths. Zhang et al. [39] (HetGNN) introduced a random walk sampler and aggregators to sample and aggregate heterogeneous neighbors for each node. The other school [11, 15, 18, 24, 35, 36] focuses on defining specific paths to capture the corresponding semantic information. Jin et al. [15] (NIRec) used sampled paths to construct the neighborhoods and designed the interaction model to capture the interactive patterns. Hu et al. [11] (MCRec) developed a co-attention mechanism to consider the mutual effect between paths.

Connections and Difference to Existing Work. We note that there is another line of edge prediction frameworks called SEAL [40] and following FakeEdge [3] which also remove the supervision edge from the sampled subgraphs. Also, there is a family of work, i.e., DropEdge [20] and DropNode [5]) that randomly remove some edges or nodes from the input graph to enhance the generalizability of graph neural networks.

Different from EMPIRE, SEAL and FakeEdge formulate the edge prediction task as the graph classification task, which we argue, is non-trivial to extend these methods to heterogeneous graph-based recommendations due to their high computation costs and the difficulty of defining the proper subgraphs regarding various metapaths. Our approach is also different from DropEdge and DropNode, because EMPIRE is not just removing some edges from the topology but also using the rest edges (called topology edges) to predict these edges (called supervision edges), thus guaranteeing the consistency of the training and test processes. In other words, each training edge is either used or dropped in [20], but is either used as the topology or the supervision in EMPIRE.

We also note that our solution of letting message-passing aware of the nodes to predict is developed based on our edge splits, and thus is naturally different from previous work [32].

2.2 MPNN based Formulation and Challenges

Our paper is developed along the line of a series work that (heterogeneous) graph based recommendation methods developed upon the message passing mechanism (i.e., MPNNs) such as [9, 31, 39].

DEFINITION 2. (MPNN BASED EDGE PREDICTION FRAMEWORK) *Considering a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and assuming that each node v_n has a feature vector $\mathbf{x}_n$, the n -th row of the feature matrix $\mathbf{X}$, we can build an arbitrary MPNN model (denoted as $\text{MPNN}(\cdot)$) to encode the embedding vector for each node. In other words, we have $\mathbf{H} = \text{MPNN}(\mathcal{G}, \mathbf{X})$, where $\mathbf{H}$ is the embedding matrix. The learned node embeddings are directly plugged into a score function (denoted as $S(\cdot)$) to obtain the final prediction of the edge existence of each pair of nodes $\langle u_s, i_t \rangle$ (denoted as $\hat{y}_{st}$), which can be formulated as*

$$\hat{y}_{st} = S(\text{MPNN}(\mathcal{G}, \mathbf{X})_s, \text{MPNN}(\mathcal{G}, \mathbf{X})_t). \quad (1)$$

One widely adopted loss is a log loss $\mathcal{L}$ defined as

$$\mathcal{L} = - \sum_{\langle u_s, i_t \rangle \in \mathcal{U} \times \mathcal{I}} \left(y_{st} \log \hat{y}_{st} + (1 - y_{st}) \log(1 - \hat{y}_{st}) \right), \quad (2)$$

where y_{st} is the label for node pair $\langle u_s, i_t \rangle$.

To be more specific, we first review how MPNN models work as follows. The core idea of MPNN is the message passing mechanism [7], where the message passing runs multiple rounds on graph to update the node embeddings. Formally, the embedding vector $\mathbf{h}_n^{(l+1)}$ of node v_n in iteration $l + 1$ is updated by

$$\mathbf{h}_n^{(l+1)} = \mathcal{U}(\mathbf{h}_n^{(l)}, \{\mathbf{m}_{n \leftarrow w}^{(l)} | w \in \mathcal{N}_{\mathcal{E}}(n)\}), \quad (3)$$

where $\mathcal{U}(\cdot)$ denotes the node embedding update function, $\mathbf{m}_{n \leftarrow w}^{(l)} = \mathcal{M}(\mathbf{h}_n^{(l)}, \mathbf{h}_w^{(l)})$ denotes the message passing from v_w to v_n , generated by a message generation function. $\mathcal{N}_{\mathcal{E}}(n)$ denotes a set of neighbor nodes connecting to v_n via the edges in $\mathcal{E}$, and the initial embedding is set as $\mathbf{h}_n^{(0)} = \mathbf{x}_n$. The above framework has derived many popular variants in terms of different designs of $\mathcal{U}(\cdot)$ and $\mathcal{M}(\cdot)$ such as GCN [17], GraphSAGE [8], GAT [27], GIN [33].

Notice that these methods are originally proposed for homogeneous graphs, we describe the concepts associated with heterogeneous graphs and how to extend these methods as follows.

A heterogeneous graph $\mathcal{G}$ is also associated with a node type mapping function $\varphi : \mathcal{V} \rightarrow \mathcal{W}$ and an edge type mapping function $\psi : \mathcal{E} \rightarrow \mathcal{R}$. $\mathcal{W}$ and $\mathcal{R}$ are the sets of pre-defined node types and edge types, where $|\mathcal{W}| + |\mathcal{R}| > 2$. As there are multiple types of edges, two nodes can be connected via different metapaths [26] defined as follows. A metapath Φ is defined as a path in the form of $W_1 \xrightarrow{R_1} W_2 \xrightarrow{R_2} \dots \xrightarrow{R_l} W_{l+1}$ that describes a composite relation $R = R_1 \odot R_2 \odot \dots \odot R_l$ between node type W_1 and W_l and $\odot$ denotes the composition operator on the edge types. Based on a metapath Φ , we can tailor the neighborhood $\mathcal{N}_{\mathcal{E}}(n)$ with the metapath based neighborhood $\mathcal{N}_{\mathcal{E}}^{\Phi}(n)$: a set of nodes connecting to v_n via metapath Φ . Taking Figure 1 as an example, we can define three metapaths $\Phi_1 = \text{User-Type}$, $\Phi_2 = \text{User-Type-Item}$, $\Phi_3 = \text{User-Type-User}$. In this case, we can derive $\mathcal{N}_{\mathcal{E}}^{\Phi_1}(u_A) = \{a_A, a_B\}$, $\mathcal{N}_{\mathcal{E}}^{\Phi_2}(u_A) = \{i_A\}$, $\mathcal{N}_{\mathcal{E}}^{\Phi_3}(u_A) = \{u_B\}$. Therefore, one can apply MPNNs to heterogeneous graphs by replacing $\mathcal{N}_{\mathcal{E}}(n)$ with $\mathcal{N}_{\mathcal{E}}^{\Phi}(n)$ in Eq. (3).

From the above description, we know that MPNN is a general framework based on the topology. Given an original graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, a principle data preprocessing procedure would first remove the test edges $\mathcal{E}_{\text{tes}}$ from the graph is to construct a training graph $\mathcal{G}_{\text{tra}} = (\mathcal{V}, \mathcal{E}_{\text{tra}})$ (and $\mathcal{E} = \mathcal{E}_{\text{tra}} \cup \mathcal{E}_{\text{tes}}$).

This setting is proposed to avoid the label leakage issue, but also causes a new challenge that is not aware of in previous literature: *misalignment of the settings of training and test process of the edge prediction task*. Concretely, the MPNN models are trained on $\mathcal{G}_{\text{tra}}$ to recover $\mathcal{E}_{\text{tra}}$ where $\mathcal{G}_{\text{tra}}$ is a graph containing the edges in $\mathcal{E}_{\text{tra}}$. In other words, during training, each training edge is simultaneously used as the topology and the supervision; while each test edge is only used as the supervision. This issue would encourage the model to over-fit $\mathcal{G}_{\text{tra}}$ and thus hold low generalizability to $\mathcal{G}$ (i.e., $\mathcal{E}_{\text{tes}}$).

Another challenge is *misalignment of the objects in the message passing mechanism and the edge prediction task*. Specifically, the

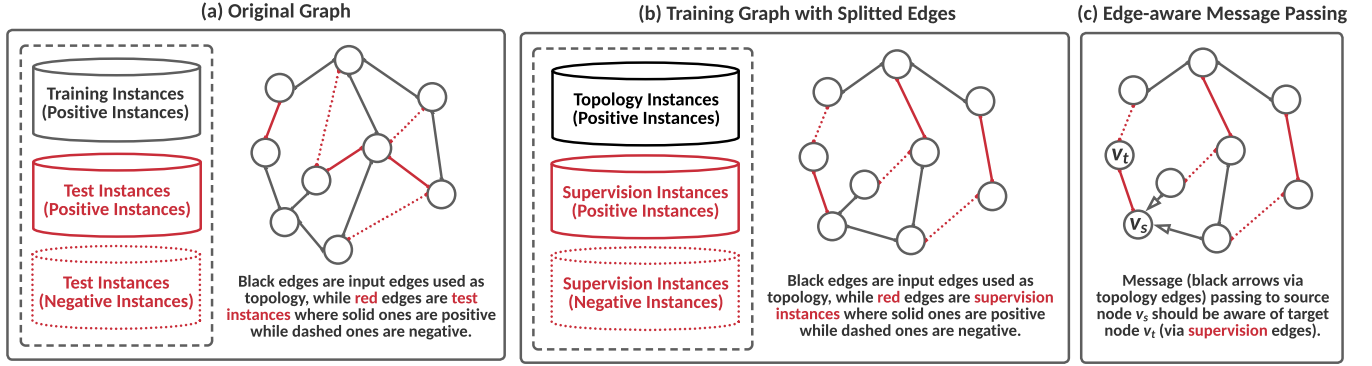


Figure 2: An illustrated example of the proposed framework EMPIRE for the edge prediction task. Given an original graph, as shown in (a), we first remove the test edges to form the training graph to avoid the label leakage issue. Then, to align the training and test processes, we propose to split the training edges into the topology edges and supervision edges, as shown in (b), where each training edge is used as either topology or supervision. On the basis of the splitted edges, we develop a novel edge-aware passing mechanism to make the messages to the source nodes being aware of the target nodes, as shown in (c).

message passing mechanism (as shown in Eq. (3)) aggregates the neighborhood information of each node to update its representation vector; while the object of the edge prediction task (as shown in Eq. (1)) is each pair of nodes. Therefore, existing message passing mechanism focuses on individual node; however, the prediction objective requires it to optimize over node pairs.

To address the above challenges, we propose EMPIRE working on the training phase of MPNN based on edge prediction framework.

3 THE EMPIRE FRAMEWORK

3.1 Elaborated Use of Training Edges

To begin with, to align the settings of training and test processes, we need to elaborate the use of each training edge to make it solely use as either the topology or the supervision instead of both. To achieve this, we introduce an edge splitter to split the training edges into two subsets: topology edges (denoted as $\mathcal{E}_{\text{top}}$) that are used as the inputs to build the graph to run the message passing; and supervision edges (denoted as $\mathcal{E}_{\text{sup}}$) that are used as the outputs to supervise the models. After that, we would randomly sample some unconnected pairs as negative instances which are together with $\mathcal{E}_{\text{sup}}$ to constitute the supervision instances (denoted as $\mathcal{D}_{\text{sup}}$). As a consequence, as depicted in Figure 2(a)(b), during either training or test processes, each edge is solely used as the topology (i.e., edges colored in black: $\mathcal{E}_{\text{tra}}$ during test and $\mathcal{E}_{\text{top}}$ during training) or the supervision (i.e., edges colored in red: $\mathcal{E}_{\text{tes}}$ during test and $\mathcal{E}_{\text{sup}}$ during training). In other words, our edge splitter addresses the first challenge introduced in Section 2.2.

While practice, we typically sample the splits by randomly assigning each training edge to $\mathcal{E}_{\text{sup}}$ with some probability $\eta \in (0, 1)$, and the rest edges are assigned to $\mathcal{E}_{\text{top}}$.

3.2 Edge-aware Message Passing

Recall that in the message passing mechanism proposed in [7] (i.e., Eq. (3)), the message generated from its neighbor node v_w to v_n can be formulated as $\mathbf{m}_{n \leftarrow w}^{(l)} = \mathbf{M}(\mathbf{h}_n^{(l)}, \mathbf{h}_w^{(l)})$: the messages to each node are solely determined by its previous embedding vector and its neighbor node. This mechanism fits the node classification task well

since the prediction over each node only depends on its property. However, directly applying this mechanism to the edge prediction might lead to a sub-optimal solution, because the prediction over each node pair needs to consider both the source and target nodes' properties (corresponding to the second challenge in Section 2.2). To this end, we need to tailor the message generation function $\mathbf{M}(\cdot)$ into a form $\mathbf{m}_{n \leftarrow w|t}^{(l)} = \mathbf{M}(\mathbf{h}_n^{(l)}, \mathbf{h}_w^{(l)}, \mathbf{h}_t^{(l)})$. However, there would be a large number of target nodes for each source node, and thus it is computationally consuming and even cause infeasibility to generate the messages for each node pair $\langle v_n, v_t \rangle$.

To make it scalable, when generating the message from neighbor node v_w to the source node v_n , we propose to model the influence of all the target nodes (i.e., v_t s) together as

$$\mathbf{m}_{n \leftarrow w}^{(l)} = \mathbf{M}(\mathbf{h}_n^{(l)}, \mathbf{h}_w^{(l)}, \{\mathbf{h}_t^{(l)} | v_t \in \mathcal{N}_{\mathcal{E}_{\text{sup}}}(n)\}), \quad (4)$$

where $\mathcal{N}_{\mathcal{E}_{\text{sup}}}(n)$ represents the set of the target nodes connecting to v_n by the supervision edges. With the generated message $\mathbf{m}_{n \leftarrow w}^{(l)}$, we then can update the representation vector $\mathbf{h}_n^{(l+1)}$ of each source node by incorporating Eq. (4) into Eq. (3):

$$\mathbf{h}_n^{(l+1)} = \mathbf{U}(\mathbf{h}_n^{(l)}, \{\mathbf{m}_{n \leftarrow w}^{(l)} | v_w \in \mathcal{N}_{\mathcal{E}_{\text{top}}}(n)\}), \quad (5)$$

where $\mathcal{N}_{\mathcal{E}_{\text{top}}}(n)$ denotes a set of neighbor nodes connecting to v_n by the topology edges. Notice that the designs of $\mathbf{M}(\cdot)$ also vary for different MPNN architectures. For clarity, we use GAT [27] as an example of homogeneous graphs.

In the context of GAT architecture, the original version of $\mathbf{M}(\cdot)$ can be written as

$$\mathbf{m}_{n \leftarrow w}^{(l)} = \alpha_{n \leftarrow w}^{(l)} \mathbf{h}_w^{(l)}, \quad (6)$$

where $\alpha_{n \leftarrow w}^{(l)}$ is a learnable attention weight to make the message dependent on the affinity between nodes v_n and v_w , i.e., passing more messages from the neighbor nodes similar to the source node. $\alpha_{n \leftarrow w}^{(l)}$ is computed as

$$\alpha_{n \leftarrow w}^{(l)} = \frac{\exp(\text{LeakyReLU}(\mathbf{w}^\top [\mathbf{h}_n^{(l)} \parallel \mathbf{h}_w^{(l)}]))}{\sum_{v_s \in \mathcal{N}_{\mathcal{E}_{\text{top}}}(n)} \exp(\text{LeakyReLU}(\mathbf{w}^\top [\mathbf{h}_n^{(l)} \parallel \mathbf{h}_s^{(l)}]))}, \quad (7)$$

Algorithm 1: EMPIRE

INPUT: training graph $\mathcal{G}_{\text{tra}} = (\mathcal{V}, \mathcal{E}_{\text{tra}})$, node feature X

- 1: **repeat**
- 2: Obtain $\mathcal{E}_{\text{top}}, \mathcal{E}_{\text{sup}}$ by randomly splitting $\mathcal{E}_{\text{tra}}$
- 3: Obtain negative supervision instances in $\mathcal{D}$ via Algorithm 3
- 4: Modify message passing generation $M(\cdot)$ using Eq. (9)
- 5: Modify updating function $U(\cdot)$ using Eq. (13)
- 6: Produce $\hat{y}_{st}$ for each $\langle u_s, i_t \rangle$ in $\mathcal{D}$ using Eq. (1)
- 7: Update parameters by minimizing L using Eq. (2)
- 8: **until** convergence

where $\parallel$ represents the concatenation operation, and $\mathbf{w}$ is trainable vector. These messages are later used to learn the representation of the source node v_n by $U(\cdot)$ (in Eq. (5)) implemented as

$$\mathbf{h}_n^{(l+1)} = \sigma \left(\mathbf{W}^{(l)} \mathbf{h}_n^{(l)} + \sum_{v_w \in \mathcal{N}_{\mathcal{E}_{\text{top}}}(n)} \mathbf{m}_{n \leftarrow w}^{(l)} \right), \quad (8)$$

where σ is an activation function, and $\mathbf{W}$ is the trainable matrix.

Based on the layer-wise propagation rule of GAT introduced above, to empower the GAT with the proposed technique, we modify its $M(\cdot)$ (i.e., Eq. (6)) following Eq. (4) as

$$\mathbf{m}_{n \leftarrow w}^{(l)} = \alpha_{n \leftarrow w}^{(l)} \left(\mathbf{h}_w^{(l)} + \sum_{v_t \in \mathcal{N}_{\mathcal{E}_{\text{sup}}}(n)} \beta_{w|t}^{(l)} \mathbf{h}_w^{(l)} \right), \quad (9)$$

where $\beta_{w|t}^{(l)}$ is another learnable attention weight to make the message aware of the target node v_t , e.g., passing more messages from the neighbor nodes related to v_t . $\beta_{w|t}^{(l)}$ is calculated as

$$\beta_{w|t}^{(l)} = \frac{\exp \left(\text{LeakyReLU}(\mathbf{w}^\top [\mathbf{h}_t^{(l)} \parallel \mathbf{h}_w^{(l)}]) \right)}{\sum_{v_s \in \mathcal{N}_{\mathcal{E}_{\text{sup}}}(n)} \exp \left(\text{LeakyReLU}(\mathbf{w}^\top [\mathbf{h}_s^{(l)} \parallel \mathbf{h}_w^{(l)}]) \right)}. \quad (10)$$

We then feed $\mathbf{m}_{n \leftarrow w}^{(l)}$ computed by Eq. (9) into Eq. (8) to complete the calculation of the representation vector of v_n in iteration $l + 1$.

As the above description is on the modification of the message generation function $M(\cdot)$, we further consider to modify the updating function $U(\cdot)$ (i.e., Eq. (5)) by using unconnected node pairs as the baseline to contrast and encourage the model to focus more on the information that can effectively distinguish connected and unconnected node pairs. To achieve this, we introduce $\gamma_{n \leftarrow w}^{(l)}$ is a learnable attention weight to measure each message $\mathbf{m}_{n \leftarrow w}^{(l)}$:

$$\mathbf{h}_n^{(l+1)} = U \left(\mathbf{h}_n^{(l)}, \{ \mathbf{m}_{n \leftarrow w}^{(l)}, \gamma_{n \leftarrow w}^{(l)} | v_w \in \mathcal{N}_{\mathcal{E}_{\text{top}}}(n) \} \right), \quad (11)$$

and $\gamma_{n \leftarrow w}^{(l)}$ is computed as

$$\gamma_{n \leftarrow w}^{(l)} = \frac{\sum_{t \in \mathcal{N}_{\mathcal{E}_{\text{sup}}}(n)} \exp \left(\text{LeakyReLU}(\mathbf{w}^\top [\mathbf{h}_t^{(l)} \parallel \mathbf{m}_{n \leftarrow w}^{(l)}]) \right)}{\sum_{t' \in \mathcal{B}(n)} \exp \left(\text{LeakyReLU}(\mathbf{w}^\top [\mathbf{h}_{t'}^{(l)} \parallel \mathbf{m}_{n \leftarrow w}^{(l)}]) \right)}. \quad (12)$$

$\mathcal{B}(n) \subseteq \mathcal{V} / \mathcal{N}_{\mathcal{E}_{\text{top}} \cup \mathcal{E}_{\text{sup}}}(n)$ is a set of randomly sampled nodes that are unconnected to source node v_n , and $|\mathcal{B}(n)| = |\mathcal{N}_{\mathcal{E}_{\text{sup}}}(n)|$. From Eq. (12), when the message is “close” to the representation vectors

of the target node and “distant” to the representation vectors of unconnected nodes, it will receive high attention.

Then, one practice of Eq. (11) is

$$\mathbf{h}_n^{(l+1)} = \sigma \left(\mathbf{W}^{(l)} \mathbf{h}_n^{(l)} + \sum_{v_w \in \mathcal{N}_{\mathcal{E}_{\text{top}}}(n)} \gamma_{n \leftarrow w}^{(l)} \mathbf{m}_{n \leftarrow w}^{(l)} \right). \quad (13)$$

3.3 Extension to Heterogeneous Graphs

As introduced in Section 2.2, the definition of the neighborhood is based on metapaths. Therefore, for each metapath Φ , we can modify Eq. (9) by replacing $\mathcal{N}_{\mathcal{E}_{\text{sup}}}(n)$ with $\mathcal{N}_{\mathcal{E}_{\text{sup}}}^\Phi(n)$. Concretely, we can derive $\mathbf{m}_{n \leftarrow w}^{\Phi(l)}$ from Eq. (9) as

$$\mathbf{m}_{n \leftarrow w}^{\Phi(l)} = \alpha_{n \leftarrow w}^{\Phi(l)} \left(\mathbf{h}_w^{(l)} + \sum_{v_t \in \mathcal{N}_{\mathcal{E}_{\text{sup}}}^\Phi(n)} \beta_{w|t}^{\Phi(l)} \mathbf{h}_w^{(l)} \right), \quad (14)$$

and similarly $\alpha_{s \leftarrow w}^{\Phi(l)}$ and $\beta_{w|t}^{\Phi(l)}$ can be computed by replacing $\mathcal{N}_{\mathcal{E}}$ with $\mathcal{N}_{\mathcal{E}}^\Phi$ in Eqs. (7) and (10). We then aggregate the messages to update the embedding vector of source node v_n as

$$\mathbf{h}_n^{(l+1)} = \sigma \left(\sum_{\Phi} \zeta^{\Phi(l)} \left(\mathbf{W}^{(l)} \mathbf{h}_n^{(l)} + \sum_{v_w \in \mathcal{N}_{\mathcal{E}_{\text{top}}}^\Phi(n)} \gamma_{n \leftarrow w}^{\Phi(l)} \mathbf{m}_{n \leftarrow w}^{\Phi(l)} \right) \right), \quad (15)$$

where $\zeta^{\Phi(l)}$ is a semantic-level attention weight introduced in [31]. $\gamma_{s \leftarrow w}^{\Phi(l)}$ is calculated by replacing $\mathcal{N}_{\mathcal{E}}$ with $\mathcal{N}_{\mathcal{E}}^\Phi$ in Eq. (12), $\zeta^{\Phi(l)}$ is calculated by normalizing the importance over all the metapaths using a softmax function as $\zeta^{\Phi(l)} = \exp(w^{\Phi(l)}) / \sum_{\Phi} \exp(w^{\Phi(l)})$ where the importance $w^{\Phi(l)}$ of metapath Φ is computed as

$$w^{\Phi(l)} = \sum_{v_n \in \mathcal{V}} \mathbf{w}^{(l)\top} \tanh \left(\mathbf{W}^{(l)} \left(\sum_{v_w \in \mathcal{N}_{\mathcal{E}_{\text{top}}}^\Phi(n)} \gamma_{n \leftarrow w}^{\Phi(l)} \mathbf{m}_{n \leftarrow w}^{\Phi(l)} \right) + \mathbf{b}^{(l)} \right). \quad (16)$$

3.4 Efficient Negative Node-Pair Sampling

As we have described our paradigm in the above subsections, we further introduce a new “hard” negative node-pair sampling technique. As depicted in Figure 2(b), to align the training process with the test process, we construct three sets of node pairs based on the training graph where the first two sets are obtained from splitting training edges: topology instances (i.e., topology edges) as topology corresponding to training instances (i.e., training edges) and positive supervision instances (i.e., supervision edges) corresponding to positive test instances (i.e., test edges); and the last one set is obtained from sampling: negative supervision instances. Note that many recent studies especially those in the information retrieval fields such as [34, 38] reveal that introducing “hard” negative instances can improve the stability and performance of the model. Therefore, we develop a new sampling trick to efficiently sample these “hard” negative instances.

Firstly, regarding the edge prediction task, we define the hard negative instances as node pairs that are somewhat likely to be

Table 1: Results of performance gain from EMPIRE on the large homogeneous graph datasets in terms of AUC, ACC, F1, LL. * indicates $p < 0.001$ in significance tests compared to DropEdge and DropNode.

MPNNs	Methods	ogb-collab				ogb-citation2			
		AUC	ACC	F1	LL	AUC	ACC	F1	LL
GCN	Origin	0.9609	0.9422	0.9431	0.1845	0.9542	0.9360	0.9382	0.2052
	DropEdge	0.9625	0.9431	0.9477	0.1801	0.9586	0.9427	0.9418	0.2004
	DropNode	0.9586	0.9387	0.9398	0.1934	0.9546	0.9375	0.9389	0.2047
	Origin ⁺	0.9634	0.9446	0.9557	0.1742	0.9591	0.9473	0.9494	0.1931
	EMPIRE ⁻	0.9678	0.9497	0.9638	0.1702	0.9675	0.9635	0.9646	0.1824
	EMPIRE	0.9703*	0.9525*	0.9651*	0.1687*	0.9685*	0.9655*	0.9663*	0.1731*
GIN	Origin	0.9645	0.9465	0.9577	0.1721	0.9578	0.9399	0.9412	0.2011
	DropEdge	0.9662	0.9488	0.9590	0.1716	0.9599	0.9480	0.9494	0.1892
	DropNode	0.9651	0.9472	0.9578	0.1719	0.9592	0.9478	0.9490	0.1900
	Origin ⁺	0.9664	0.9492	0.9597	0.1708	0.9616	0.9492	0.9509	0.1888
	EMPIRE ⁻	0.9748	0.9557	0.9628	0.1640	0.9655	0.9545	0.9536	0.1834
	EMPIRE	0.9763*	0.9575*	0.9660*	0.1622*	0.9665*	0.9548*	0.9553*	0.1820*
GAT	Origin	0.9532	0.9351	0.9387	0.2074	0.9478	0.9254	0.9276	0.2246
	DropEdge	0.9575	0.9385	0.9420	0.2026	0.9502	0.9284	0.9314	0.2137
	DropNode	0.9541	0.9363	0.9392	0.2067	0.9477	0.9257	0.9274	0.2228
	Origin ⁺	0.9616	0.9422	0.9467	0.1898	0.9556	0.9351	0.9365	0.1978
	EMPIRE ⁻	0.9672	0.9481	0.9624	0.1754	0.9623	0.9424	0.9406	0.1853
	EMPIRE	0.9676*	0.9489*	0.9645*	0.1723*	0.9642*	0.9432*	0.9416*	0.1844*
GraphSAGE	Origin	0.9612	0.9413	0.9415	0.1852	0.9321	0.9134	0.9171	0.2417
	DropEdge	0.9653	0.9458	0.9459	0.1745	0.9345	0.9162	0.9215	0.2383
	DropNode	0.9618	0.9417	0.9416	0.1847	0.9320	0.9138	0.9170	0.2410
	Origin ⁺	0.9655	0.9457	0.9459	0.1752	0.9384	0.9206	0.9260	0.2350
	EMPIRE ⁻	0.9711	0.9601	0.9503	0.1700	0.9448	0.9259	0.9324	0.2291
	EMPIRE	0.9714*	0.9604*	0.9502*	0.1687*	0.9443*	0.9258*	0.9347*	0.2279*

connected, but not connected in the training graph. Hence, a direct approach is to apply an edge prediction model over all unconnected node pairs, and select several node pairs with the high prediction scores as the hard negative instances. However, one obvious drawback of this approach is unacceptable computation costs.

To address this issue, we propose to generate the hard negative instances by using the non-parametric common-neighbor heuristic [1]. As shown in the edge prediction leaderboard[†] on Open Graph Benchmark (OGB), the common-neighbor heuristic can be regarded as a strong edge prediction model.

We further develop a new retrieval algorithm to find all the node pairs containing the common neighbors with $O(|V|^2)$ computations. Considering that it might still be unacceptable in some real-world cases, we present an uniform sampling version to sample n_{cn} node pairs with $O(|V| + |\mathcal{E}_{top}| + n_{cn})$ computations. Refer to Appendix A for the pseudo-code and analysis of these algorithms.

3.5 Overall Algorithm

We summarize the proposed EMPIRE paradigm in Algorithm 1. We note that during each split, EMPIRE is trained over a subset of training samples; however, we can repeatedly split edges and train EMPIRE. Therefore, the training pool of EMPIRE can cover all the training samples.

We can see that there are four operations in EMPIRE: randomly splitting edges in line 2 costs $O(|\mathcal{E}_{tra}|)$ computation, and building

the supervision instances using efficient negative node-pair sampling in line 3 costs $O(|V| + (1 + \eta)|\mathcal{E}_{tra}|)$. Let C_{MPNN} denote the time complexity of the MPNN model. Taking GAT [27] as an example, the time complexity of a single-layer GAT over F' dimension would be expressed as $C_{MPNN} = O(|V|F'F + |\mathcal{E}_{tra}|F')$ where F is the number of input features. And, the message generation in line 4 takes $O(|\mathcal{E}_{top}||\mathcal{E}_{sup}|F')$ computation, and the time complexity of updating in line 5 is $O(|V|F'F + |\mathcal{E}_{sup}|F')$. The overall complexity is $O(|V|F'F + |\mathcal{E}_{top}||\mathcal{E}_{sup}|F') = O(|V|F'F + \eta(1 - \eta)|\mathcal{E}_{tra}|^2F')$.

4 EXPERIMENT

4.1 Datasets and Experimental Flow

Dataset Description As exemplified in Section 3, the proposed framework EMPIRE can be applied to both homogeneous and heterogeneous graph based recommendations. For homogeneous graph datasets, we select two benchmark datasets from OGB [12]: **ogbl-collab** is an author collaboration graph with 235,868 nodes and 2,464,517 edges, where the task is to recommend potential collaborators (i.e., authors) to authors; **ogbl-citation2** is a paper citation network with 2,927,963 nodes and 30,561,187 edges, where the task is to recommend potential references (i.e., papers) to papers. Besides the above homogeneous graph datasets, we include four heterogeneous graph benchmark datasets: **LastFM** [10] is an audio recommendation network with 2,101 user nodes, 18,746 artist nodes, 12,648 tag nodes, 92,834 user-artist edges, 25,434 user-user edges, 186,479 user-tag edges, where the task is to recommend

[†]https://ogb.stanford.edu/docs/leader_linkprop/

artists to users; **Amazon** [19] is an e-commerce recommendation network with 6,170 user nodes, 2,753 item nodes, 3,857 view nodes, 334 brand nodes, 146,230 user-item edges, 5,694 item-view edges, 2,753 item-brand edges, 37,958 user-user edges, where the task is to recommend items to users; **Yelp** [10] is a business-based social network with 16,239 user nodes, 14,284 business nodes, 47 city nodes, 511 genre nodes, 198,397 user-business edges, 158,590 user-user edges, 76,875 user-city edges, 14,267 business-city edges, 40,009 business-genre edges, where the task is to recommend businesses to users; **Douban** [41] is a book recommendation network with 13,024 user nodes, 22,347 book nodes, 2,936 group nodes, 10,805 author nodes, 1,815 publisher nodes, 598,420 user-book edges, 169,150 user-user edges, 1,189,271 user-group edges, 21,907 book-author edges, 21,773 book-publisher edges, where the task is to recommend books to users. We evaluate the performance in terms of Area Under ROC Curve (AUC), Accuracy (ACC), F1 score (F1), and Logloss (LL). The threshold for positive prediction is set as 0.5. For each dataset, we randomly split the edges into training, validation, and test sets with a ratio of 6:2:2, and hyper-parameters are selected from validation.

Baseline Description As EMPIRE is a plug-in method that can be seamlessly incorporated into various MPNN models on both homogeneous and heterogeneous graph datasets, we introduce the following MPNN models as backbones. For homogeneous graph models, we use: **GCN** [17] is a graph convolutional network particularly designed for graph structured data; **GIN** [33] combines MLP with GCN that is provably as powerful as the Weisfeiler-Lehman graph isomorphism test; **GAT** [27] combines an attention mechanism with GCN to model the different contributions from neighbor nodes; **GraphSAGE** [8] is a graph neural network following a sample-aggregation paradigm. For heterogeneous graph models, we use: **RGCN** [21] combines graph neural networks with factorization models for edge prediction task; **HAN** [31] is a hierarchical attention network designed on heterogeneous graphs to capture node- and semantic-level information; **HetGNN** [39] is a heterogeneous version of GCN to handle the heterogeneity in node feature and graph structure. **PinSage** [34] is a deployed version of SAGE. As PinSage is originally proposed for homogeneous graphs, we extend PinSage to heterogeneous graphs by using metapaths in HAN to define neighborhoods.

To illustrate the performance gain from EMPIRE, we introduce following variants and baseline methods DropEdge [20] and DropNode [5], all of which are developed based on Origin:

- **DropEdge** randomly removes a certain number of training edges acting as a data augementer and also a message reducer.
- **DropNode** randomly removes a certain number of nodes acting as a data augementer and also a message reducer.
- **Origin⁺** applies an edge splitter to randomly sample some training edges as supervision and use the others as topology.
- **EMPIRE** is our proposed method.
- **EMPIRE⁻** is a variant of EMPIRE that does not use hard negative node pairs.

Implementation Details. We employ (DGL) [29] to realize all the above MPNN models. We specify some key hyper-parameter settings as follows. To make fair comparisons, all the MPNN models

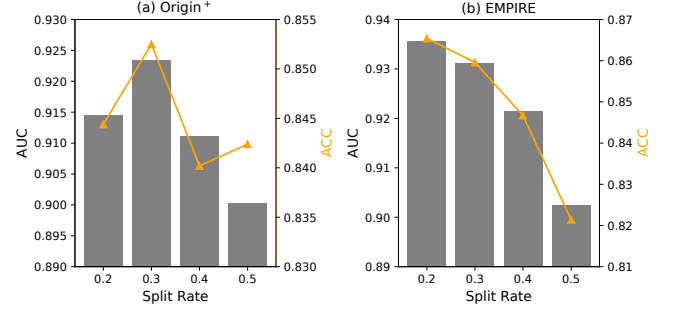


Figure 3: Performance Comparisons of Origin⁺ and EMPIRE using HAN as the basic MPNN models under different split rate η on Amazon dataset in terms of AUC and ACC.

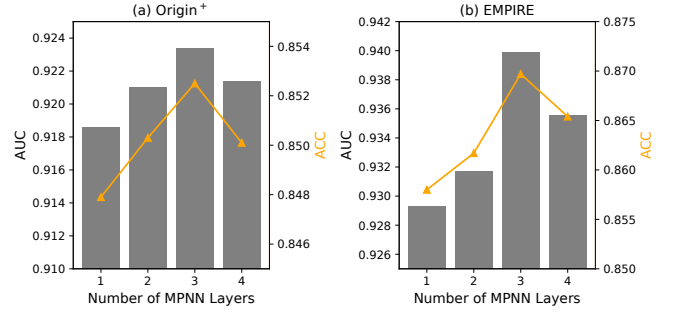


Figure 4: Performance Comparisons of Origin⁺ and EMPIRE using HAN as the basic MPNN models under different numbers of propagation layers on Amazon dataset in terms of AUC and ACC.

have the same three layers, the dimension size of the embeddings is set to 128, and the learning rate space is $[0.001, 0.0001]$. The dropout rate of input is set to 0.6, of edge is set to 0.3. We use the Adam optimizer for all the models. The models are trained under the same hardware settings with an Amazon EC2 p3.8xlarge instance[‡].

We run each method 10 times and report the average results of each metric. All the codes will be released at publication time.

4.2 Performance Comparison

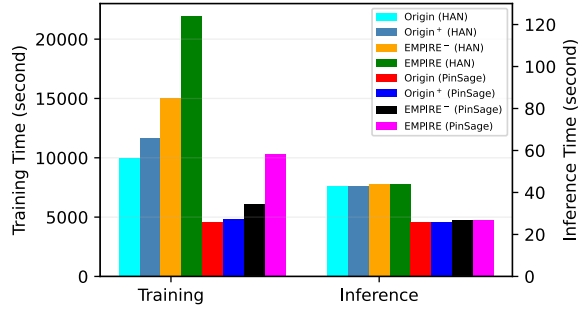
Tables 1 and 2 summarize all the main results. From tables, we can see that the performance gains from our techniques vary with different basic MPNN models. We list the major findings as follows.

- Results of comparisons between Origin and Origin⁺ demonstrate the superiority of our proposed edge splitter, i.e., these performance gains can be attributed to specifying the use of each edge. We note that this technique is super simple yet can consistently bring significant improvements in homogeneous graph and heterogeneous graph datasets.
- Comparisons between Origin⁺ and EMPIRE⁻ illustrate the performance gain brought from our proposed edge-aware message passing mechanism. We can find that although our new edge-aware message passing mechanism works well on multiple MPNN models, it can bring more improvements on GCN, GIN, GAT, RGCN,

[‡]Detailed setting of AWS E2 instance can be found at https://aws.amazon.com/ec2/instance-types/?nc1=h_ls.

Table 2: Results of performance gain from EMPIRE on the heterogeneous graph datasets in terms of AUC, ACC, F1. * indicates $p < 0.001$ in significance tests compared to DropEdge and DropNode.

MPNNs	Methods	LastFM			Amazon			Yelp			Douban		
		AUC	ACC	F1	AUC	ACC	F1	AUC	ACC	F1	AUC	ACC	F1
RGCN	Origin	0.8532	0.8382	0.8354	0.8120	0.7403	0.7322	0.8879	0.8411	0.8330	0.9203	0.8412	0.8267
	DropEdge	0.8617	0.8440	0.8398	0.8129	0.7413	0.7333	0.8909	0.8435	0.8365	0.9287	0.8486	0.8315
	DropNode	0.8597	0.8412	0.8378	0.8119	0.7406	0.7326	0.8900	0.8426	0.8349	0.9220	0.8425	0.8287
	Origin ⁺	0.8624	0.8447	0.8412	0.8125	0.7496	0.7346	0.8935	0.8504	0.8434	0.9351	0.8573	0.8394
	EMPIRE ⁻	0.8719	0.8550	0.8552	0.8208	0.7557	0.7490	0.9144	0.8616	0.8494	0.9455	0.8645	0.8496
	EMPIRE	0.8732*	0.8564*	0.8591*	0.8263*	0.7575*	0.7581*	0.9181*	0.8684*	0.8562*	0.9565*	0.8695*	0.8533*
HAN	Origin	0.8915	0.8736	0.8696	0.9156	0.8488	0.8427	0.8487	0.8082	0.7972	0.9243	0.8500	0.8451
	DropEdge	0.8997	0.8798	0.8744	0.9210	0.8512	0.8451	0.8502	0.8109	0.7999	0.9290	0.8587	0.8491
	DropNode	0.8912	0.8735	0.8695	0.9167	0.8491	0.8430	0.8506	0.8113	0.8005	0.9267	0.8521	0.8460
	Origin ⁺	0.9086	0.8817	0.8758	0.9234	0.8525	0.8496	0.8527	0.8153	0.8025	0.9286	0.8586	0.8488
	EMPIRE ⁻	0.9183	0.8823	0.8735	0.9310	0.8621	0.8598	0.8677	0.8265	0.8186	0.9376	0.8665	0.8611
	EMPIRE	0.9194*	0.8835*	0.8772*	0.9356*	0.8654*	0.8623*	0.8714*	0.8335*	0.8242*	0.9403*	0.8702*	0.8642*
HetGNN	Origin	0.7936	0.7858	0.7777	0.7744	0.7108	0.7109	0.8683	0.8297	0.8205	0.8737	0.7912	0.7915
	DropEdge	0.8145	0.8010	0.7903	0.7902	0.7318	0.7321	0.8727	0.8353	0.8325	0.9286	0.8586	0.8488
	DropNode	0.8034	0.7902	0.7813	0.7743	0.7110	0.7115	0.8627	0.8287	0.8152	0.9286	0.8586	0.8488
	Origin ⁺	0.8212	0.8140	0.8111	0.7912	0.7326	0.7332	0.8783	0.8394	0.8387	0.9203	0.8412	0.8267
	EMPIRE ⁻	0.8389	0.8276	0.8277	0.8018	0.7411	0.7408	0.8906	0.8510	0.8498	0.9313	0.8603	0.8556
	EMPIRE	0.8392	0.8285	0.8282	0.8107	0.7502	0.7489	0.8921	0.8516	0.8523	0.9463	0.8732	0.8642
PinSage	Origin	0.8329	0.8200	0.8179	0.8653	0.8046	0.8004	0.8021	0.7682	0.7567	0.8264	0.7532	0.7368
	DropEdge	0.8345	0.8227	0.8208	0.8698	0.8104	0.8099	0.8096	0.7713	0.7625	0.8287	0.7549	0.7387
	DropNode	0.8336	0.8209	0.8192	0.8668	0.8051	0.8008	0.8104	0.7724	0.7635	0.8265	0.7535	0.7371
	Origin ⁺	0.8348	0.8229	0.8215	0.8712	0.8118	0.8105	0.8120	0.7741	0.7659	0.8295	0.7577	0.7404
	EMPIRE ⁻	0.8361	0.8249	0.8232	0.8789	0.8175	0.8158	0.8243	0.7812	0.7723	0.8309	0.7586	0.7412
	EMPIRE	0.8372*	0.8262*	0.8249*	0.8802*	0.8212*	0.8209*	0.8264*	0.7835*	0.7742*	0.8312*	0.7589*	0.7417*

**Figure 5: Comparisons of training and inference time of EMPIRE and its variants (Origin⁺ and EMPIRE⁻) using HAN and PinSage as the basic MPNN models on Amazon dataset.**

HAN, HetGNN than GraphSAGE, PinSage. One possible explanation is that GraphSAGE and PinSage are randomly sampling the neighborhood whose performance partly relies on randomness and makes it more difficult for EMPIRE⁻ to learn stable attention weights. In other words, “unstable” backbone MPNN models might lead to “unstable” improvements of our plug-in methods.

- Comparisons between EMPIRE⁻ and EMPIRE show the performance gain of our “hard” negative node pairs. We can see that this technique would consistently bring improvements and its performance largely depends on what kind of datasets are used in the experiments. If considering the datasets with a large number of unconnected node pairs (i.e., graphs are sparser) such as Yelp,

our method would perform better, compared to the other datasets such as Amazon.

- Differences between Origin⁺ and baseline methods (i.e., DropNode, DropEdge) show that our splitting technique could outperform randomly dropping nodes and edges in most datasets. One explanation is that our technique does not only randomly some edges from the topology but also use them as supervision, which could make each training edge solely used as topology or supervision, aligning with the test process.

4.3 Hyperparameter Study

Impact of Split Rate. As our edge splitter is splitting the training edges into the topology and supervision edges which is the basis of our edge-aware message passing mechanism, it is essential to investigate the impact of split rate η . From the description in Section 3.1, we can see: when η approaches 0, there is no edge used as supervision, which is meaningless; while when η tends to 1, it represents the case where there is no input graph topology which also tells nothing. Hence, we select 0.2, 0.3, 0.4, 0.5 as the values of η to depict the trend of performance of Origin⁺ and EMPIRE using HAN as the basic MPNN model in terms of AUC and ACC on Amazon dataset. As Figure 3 shows, the best performance of either Origin⁺ or EMPIRE occurs around 0.2 to 0.3, showing that 70% to 80% training edges used as the topology and the rest 30% to 20% training edges used as the supervisions would be the best choice. One possible reason is that a larger η means more training edges used as the supervisions and fewer edges used for the topology,

which would cause the data sparsity issue on the input graph; while a smaller η would encourage the model to focus more on the input graph, which might lead to the over-fitting issue.

Impact of Number of Propagation Layers. Another key hyper-parameter is the number of layers of propagation layers in MPNN models. Normally speaking, the best choice of the number of propagation layers of MPNN models lies among the values 1,2,3,4. Hence, we evaluate their performance of Origin⁺ and EMPIRE using HAN as the basic MPNN model in terms of AUC and ACC on Amazon dataset and depict the results in Figure 4, which shows the performance changes from the number of propagation layers are relatively small, and among those choices, 3 might be the best one.

4.4 Complexity Study

We study the time complexity of applying the proposed techniques (i.e., Origin⁺, EMPIRE⁻, EMPIRE) on Amazon dataset using HAN and PinSage as the basic MPNN models. We depict the training and inference times for one round of the whole data in Figure 5. From the figure, we can see the most time-consuming component in EMPIRE is our negative node-pair sampling algorithm, although we have proposed a new algorithm to accelerate the process. One possible solution is to pre-compute and store these node pairs into a buffer. And then, we can simply sample instances from the buffer during training, and as EMPIRE acts during the training phase, there is no extra cost for inference time.

Algorithm 2: Common Neighbor Nodes Retrieval

```

1  $\mathcal{B} \leftarrow \{\}$ 
2 for each  $v_u \in \mathcal{V}$  do
3   for each  $v_o, v_w \in \mathcal{N}_{\mathcal{E}_{\text{tra}}}(u)$  do
4     if  $\langle v_o, v_w \rangle \notin \mathcal{E}_{\text{tra}}$  then
5        $\mathcal{B} \leftarrow \mathcal{B} \cup \{(v_o, v_w)\}$ 
6     end
7   end
8 end
```

5 CONCLUSION AND FUTURE WORK

In this paper, we analyze the key differences between the node classification and edge prediction tasks, and present a novel edge prediction based recommendation framework named EMPIRE to tailor existing MPNN models (originally proposed for the node classification) to be more fit for the edge prediction, where we first propose to specify the use of training edges, and then develop a novel edge-aware message passing mechanism to make the messages to the source nodes be aware of the target nodes. We also show that sampled hard negative instances could further improve the performance. Our experimental results demonstrate that our techniques can be applied to various MPNN models. It would be interesting to incorporate them into other graph tasks (e.g., graph classification) in future work.

Algorithm 3: Common Neighbor Nodes Sampling

```

1  $\mathcal{B} \leftarrow \{\}$ 
2 for each  $v_u \in \mathcal{V}$  do
3    $c_u \leftarrow d_u(d_u - 1)$  where  $d_u$  is the degree of node  $v_u$ 
4 end
5  $c \leftarrow \sum_u c_u$ 
6 for each  $v_u \in \mathcal{V}$  do
7    $p_u \leftarrow c_u/c$ 
8 end
9 for  $i \leftarrow 1$  to  $n_{\text{cn}}$  do
10  Sample  $v_u$  with probability  $p_u$ 
11  Uniformly sample two different neighbor nodes (i.e.,  $v_o$ 
    and  $v_w$ ) of  $v_u$ 
12  if  $\langle v_o, v_w \rangle \notin \mathcal{E}_{\text{tra}}$  then
13     $\mathcal{B} \leftarrow \mathcal{B} \cup \{(v_o, v_w)\}$ 
14  end
15 end
```

A COMMON NEIGHBOR NODES RETRIEVAL AND SAMPLING ALGORITHMS

In this part, we first provide a retrieval algorithm that can search and retrieve all the node pairs satisfying the common-neighbor heuristics [1] in the training graph $\mathcal{G}_{\text{tra}}$. Equivalently, we formulate the task as to find all the two-node paths in $\mathcal{G}_{\text{tra}}$. Formally, our input is $\mathcal{G}_{\text{tra}} = (\mathcal{V}, \mathcal{E}_{\text{tra}})$ and the output is $\mathcal{B}$ containing all the node pairs (v_o, v_w) sharing the neighbor node v_u are enumerated. As the nodes v_u, v_o, v_w can be any nodes in $\mathcal{G}_{\text{tra}}$, the time complexity of Algorithm 2 is $O(|\mathcal{V}|^2)$.

However, in the real-world scenarios, there are the many cases with large graphs, where Algorithm 2 might be not practical due to its high computation costs. Therefore, we further propose a novel common neighbor nodes sampling algorithm in Algorithm 3 that can uniformly sample n_{cn} node pairs. Its inputs and outputs are identical to Algorithm 2.

Let d_u denote the degree of node v_u . For the correctness of Algorithm 3, we formulate the sampling the node pairs into uniformly sampling three-node path, namely (v_o, v_u, v_w) . We first sample the intermediate node of the path (i.e., v_u) where there are $d_u(d_u - 1)/2$ three-node paths with v_u as its intermediate node. Next, we sample v_u with the probability p_u to ensure uniformity, where $p_u = d_u(d_u - 1) / \sum_u d_u(d_u - 1)$, and then sample its two neighbor nodes to form the paths. For the time complexity, the computation for the degree requires $O(|\mathcal{E}_{\text{tra}}|)$, the first two loops require $O(|\mathcal{V}|)$, and the last loop requires $O(n_{\text{cn}})$ if we sample (v_o, v_w) using alias method [28]. So, the overall time complexity of Algorithm 3 is $O(|\mathcal{V}| + |\mathcal{E}_{\text{tra}}| + n_{\text{cn}})$.

REFERENCES

- [1] Lada A Adamic and Eytan Adar. 2003. Friends and neighbors on the web. *Social networks* (2003).
- [2] Ye Bi, Liqiang Song, Mengqiu Yao, Zhenyu Wu, Jianming Wang, and Jing Xiao. 2020. A heterogeneous information network based cross domain insurance recommendation system for cold start users. In *SIGIR*.
- [3] Kaiwen Dong, Yijun Tian, Zhichun Guo, Yang Yang, and Nitesh Chawla. 2022. FakeEdge: Alleviate Dataset Shift in Link Prediction. In *Learning on Graphs Conference*. PMLR, 56–1.
- [4] Wei Feng and Jianyong Wang. 2012. Incorporating heterogeneous information for personalized tag recommendation in social tagging systems. In *KDD*.
- [5] Wenzheng Feng, Jie Zhang, Yuxiao Dong, Yu Han, Huanbo Luan, Qian Xu, Qiang Yang, Evgeny Kharlamov, and Jie Tang. 2020. Graph random neural networks for semi-supervised learning on graphs. *Neurips* (2020).
- [6] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Lindaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. 2018. Cypher: An evolving query language for property graphs. In *ICMD*.
- [7] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *ICML*.
- [8] William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *NIPS*.
- [9] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *SIGIR*.
- [10] Binbin Hu, Chuan Shi, Wayne Xin Zhao, and Tianchi Yang. 2018. Local and global information fusion for top-n recommendation in heterogeneous information network. In *CIKM*.
- [11] Binbin Hu, Chuan Shi, Wayne Xin Zhao, and Philip S Yu. 2018. Leveraging meta-path based context for top-n recommendation with a neural co-attention model. In *KDD*.
- [12] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open graph benchmark: Datasets for machine learning on graphs. In *Technical report arXiv:2005.00687*.
- [13] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous graph transformer. In *WWW*.
- [14] Jin Huang, Wayne Xin Zhao, Hongjian Dou, Ji-Rong Wen, and Edward Y Chang. 2018. Improving sequential recommendation with knowledge-enhanced memory networks. In *SIGIR*.
- [15] Jiarui Jin, Jiarui Jin, Yuchen Fang, Kounianhua Du, Weinan Zhang, Yong Yu, Zheng Zhang, and Alexander J Smola. 2020. An efficient neighborhood-based interaction model for recommendation on heterogeneous graph. In *KDD*.
- [16] Jiarui Jin, Yangkun Wang, Kounianhua Du, Weinan Zhang, Zheng Zhang, David Wipf, Yong Yu, and Quan Gan. 2022. Inductive Relation Prediction Using Analogy Subgraph Embeddings. In *ICLR*.
- [17] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. In *ICLR*.
- [18] Chen Luo, Wei Pang, Zhe Wang, and Chenghua Lin. 2014. Hete-cf: Social-based collaborative filtering recommendation using heterogeneous relations. In *ICDM*.
- [19] Jianmo Ni, Jiacheng Li, and Julian McAuley. 2019. Justifying recommendations using distantly-labeled reviews and fine-grained aspects. In *EMNLP*.
- [20] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. 2019. Droppedge: Towards deep graph convolutional networks on node classification. *ICLR* (2019).
- [21] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *ESWC*.
- [22] Chuan Shi, Binbin Hu, Wayne Xin Zhao, and S Yu Philip. 2018. Heterogeneous information network embedding for recommendation. *TKDE* (2018).
- [23] Chuan Shi, Yitong Li, Jiawei Zhang, Yizhou Sun, and S Yu Philip. 2016. A survey of heterogeneous information network analysis. *TKDE* (2016).
- [24] Chuan Shi, Zhiqiang Zhang, Ping Luo, Philip S Yu, Yading Yue, and Bin Wu. 2015. Semantic path based personalized recommendation on weighted heterogeneous information networks. In *CIKM*.
- [25] Abhay Singh, Qian Huang, Sijia Linda Huang, Omkar Bhalerao, Horace He, Ser-Nam Lim, and Austin R Benson. 2021. Edge proposal sets for link prediction. *Technical report arXiv:2106.15810* (2021).
- [26] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S Yu, and Tianyi Wu. 2011. Pathsims: Meta path-based top-k similarity search in heterogeneous information networks. *Proceedings of the VLDB Endowment* (2011).
- [27] Petar Velićković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph attention networks. In *ICLR*.
- [28] Alastair J Walker. 1977. An efficient method for generating discrete random variables with general distributions. *TOMS* (1977).
- [29] Minjie Wang, Lingfan Yu, Da Zheng, Quan Gan, Yu Gai, Zihao Ye, Mufei Li, Jinjing Zhou, Qi Huang, Chao Ma, et al. 2019. Deep Graph Library: Towards Efficient and Scalable Deep Learning on Graphs. *Technical report* (2019).
- [30] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *SIGIR*.
- [31] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *WWW*.
- [32] Yu Wang, Yuying Zhao, Yi Zhang, and Tyler Derr. 2022. Collaboration-Aware Graph Convolutional Networks for Recommendation Systems. *arXiv preprint arXiv:2207.06221* (2022).
- [33] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *ICLR* (2018).
- [34] Rex Ying, Ruining He, Kaifeng Chen, Peng Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *KDD*.
- [35] Xiao Yu, Xiang Ren, Yizhou Sun, Quanquan Gu, Bradley Sturt, Urvashi Khandelwal, Brandon Norick, and Jiawei Han. 2014. Personalized entity recommendation: A heterogeneous information network approach. In *WSDM*.
- [36] Xiao Yu, Xiang Ren, Yizhou Sun, Bradley Sturt, Urvashi Khandelwal, Quanquan Gu, Brandon Norick, and Jiawei Han. 2013. Recommendation in heterogeneous information networks with implicit user feedback. In *RecSys*.
- [37] Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J Kim. 2019. Graph transformer networks. *Neurips* (2019).
- [38] Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2021. Optimizing dense retrieval model training with hard negatives. In *SIGIR*.
- [39] Chuxu Zhang, Dongjin Song, Chao Huang, Ananthram Swami, and Nitesh V Chawla. 2019. Heterogeneous graph neural network. In *KDD*.
- [40] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. *NIPS* (2018).
- [41] Jing Zheng, Jian Liu, Chuan Shi, Fuzhen Zhuang, Jingzhi Li, and Bin Wu. 2017. Recommendation in heterogeneous information network via dual similarity regularization. *International Journal of Data Science and Analytics* (2017).